Predicting Next-Day Rain with Classification Models in KNIME

Introduction to Data Analytics

Student Name: Brunda Varshini Katapalli

Introduction

The goal of this study is to create a model for predicting incorporating past rainfall data which is the RainTomorrow attribute. This attribute categorizes whether it will rain tomorrow using binary values like 0 for "No Rain" and 1 for "Rain."

Exploring the data to find significant trends, implementing classification algorithms into practice to forecast RainTomorrow, and assessing model performance using measures like accuracy and F1-score are some of the main goals. The project's ultimate goal is to produce a trustworthy model that accurately forecasts rainfall, enabling useful applications.

In order to solve this data mining issue, I currently have the existing dataset, WeatherData. This is a CSV file that contains some weather data attributes such as Location, MinTemp, MaxTemp, Rainfall, Evaporation, Sunshine, WindGustDir , WindGustSpeed, WindDir9am, WindDir3pm, WindSpeed9am, WindSpeed3pm, Humidity9am, Humidity3pm, Pressure9am, Pressure3pm, Cloud9am, Cloud3pm, Temp9am, Temp3pm, RainToday and RainTomorrow. The class attribute in this data mining problem is RainTomorrow. The dataset is large-scale, containing 51,199 rows of data in String and Integer datatypes. There is a need to perform extensive pre-processing to clean the data by eliminating noise, addressing missing values, and identifying any outliers that may be present.

The output of the problem includes predicting whether it will rain tomorrow based on various existing weather attributes from the dataset. The predicted outcome will include a model that classifies each row of data into one of 2 classes: RainTomorrow = 1 or RainTomorrow = 0. Thus, the output will be a model that predicts the likelihood of rain. The performance of the model will be assessed using evaluation metrics such as Accuracy, Precision, Recall, F1 score, and ROC.

Methodology

A. Data Preparation (Data Preprocessing and Data Exploration)

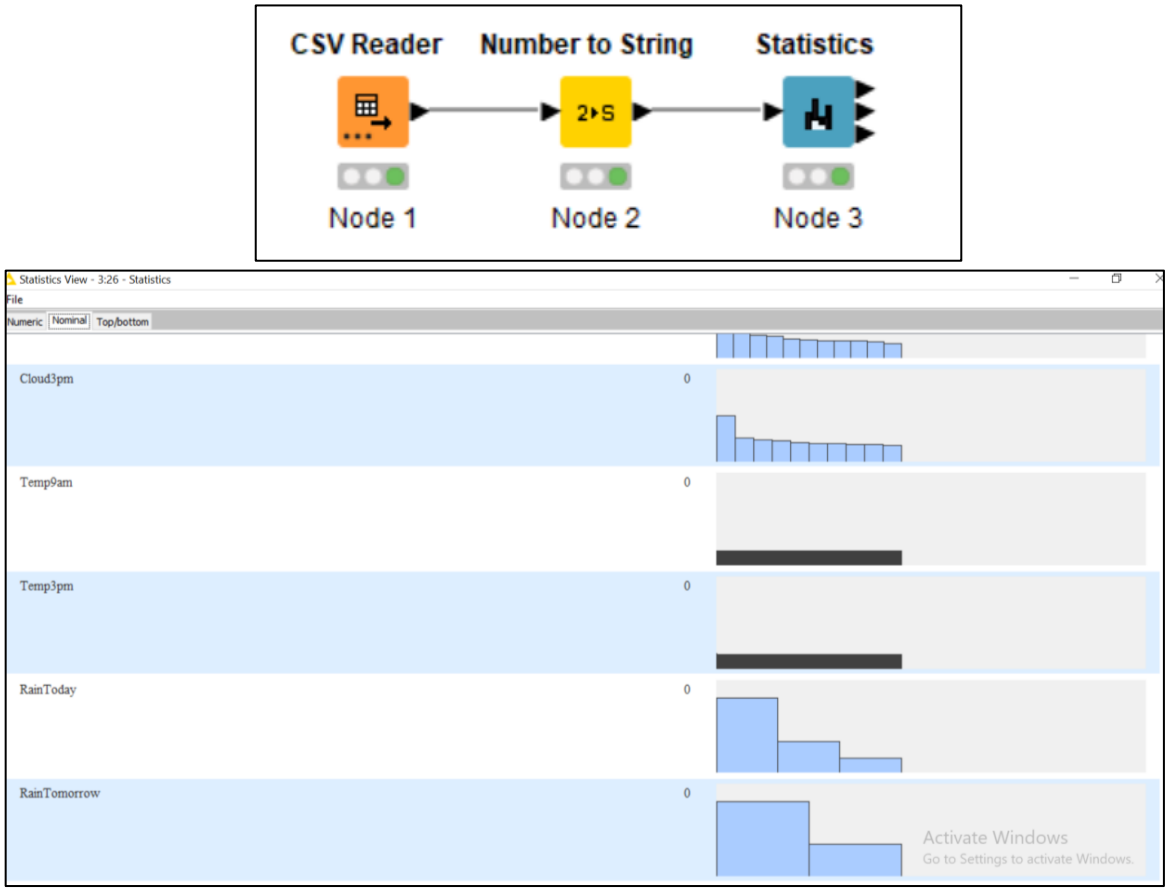


Figure 1 Statistics for WeatherData dataset

The WeatherData dataset is identified as Unbalanced data. This is based on the output from the Number to String node, which converted the class attribute, RainTomorrow attribute from an integer to a string format. The Statistics node confirmed this observation, as evident from the histograms shown in Figure 1, which illustrates the class distribution in the dataset. This finding emphasizes the necessity of careful modeling techniques, such as choosing suitable classifiers. Resolving this imbalance will improve the robustness of the model and guarantee more accurate predictions for every class.

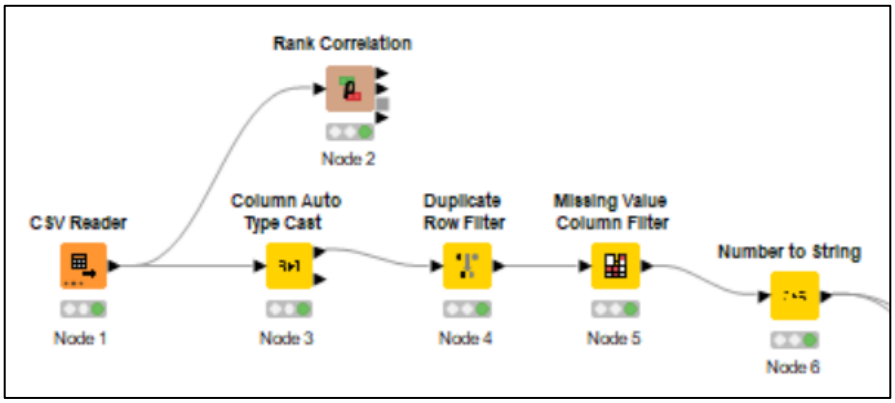


Figure 2 KNIME Workflow for Data Pre-processing and Data Exploration

After loading the data, Column Auto Type Cast node is used to automatically determine and assign the appropriate data types to each column based on the values. This step ensures accuracy of data processing.

The Duplicate Row Filter node scans the dataset for any duplicate entries, removing any rows that are identical across all selected columns. This is essential for maintaining data integrity and ensuring that analyses and model training are based on unique observations, which can prevent bias in results. Therefore, by applying this node to the dataset, the number of rows decreased from 51,199 to 51,187.

The Missing Value Column Filter node identifies columns with missing values. Addressing missing values is crucial to ensure the completeness of the dataset and to avoid potential issues during model training. The String to Number node converts the categorical variables into numerical format. This transformation is essential for effective feature encoding, allowing models to interpret the data properly.

Finally, this node scales the numerical features to a common range, typically between 0 and 1 or standardizing to have a mean of 0 and a standard deviation of 1. Normalization is important for algorithms sensitive to feature scales, such as KNN and SVM, as it ensures that all features contribute equally to the model training process.

Rank Correlation:

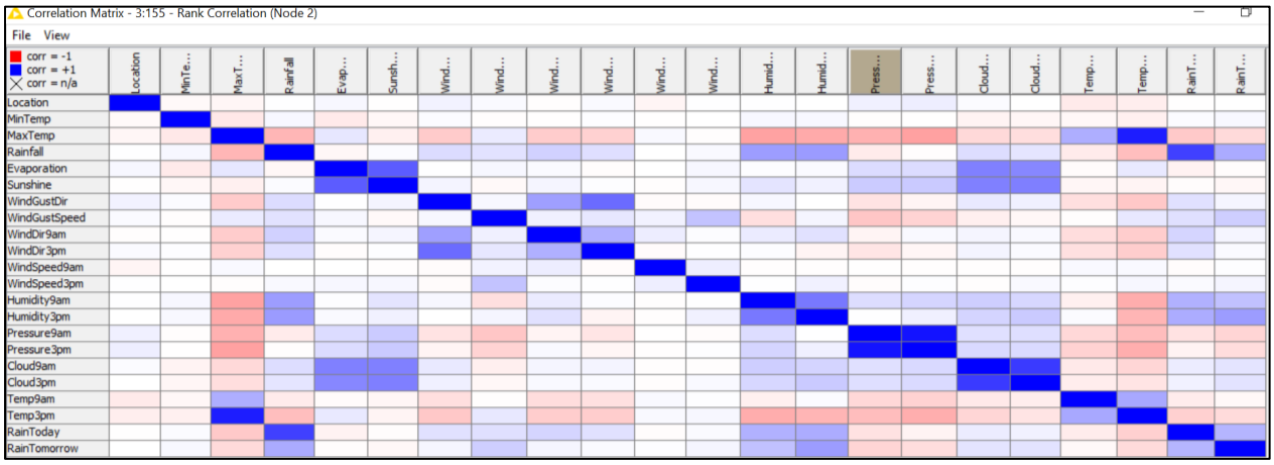


Figure 3 shows the Rank Correlation of WeatherData dataset

Rank Correlation is utilized to check how the rankings of different attributes in the dataset are related. This method helps us understand whether 2 attributes tend to increase or decrease together. By calculating the correlation for each pair of attributes, it is visible from Figure 3 that MaxTemp, WindGustDir, WindDir9am, WindDir3pm, Pressure9am,Pressure3pm,Temp9am and Temp3pm are least related to the class attribute,RainTomorrow. There are more shades of red, indicating strong negative correlations.

How the problem was solved?

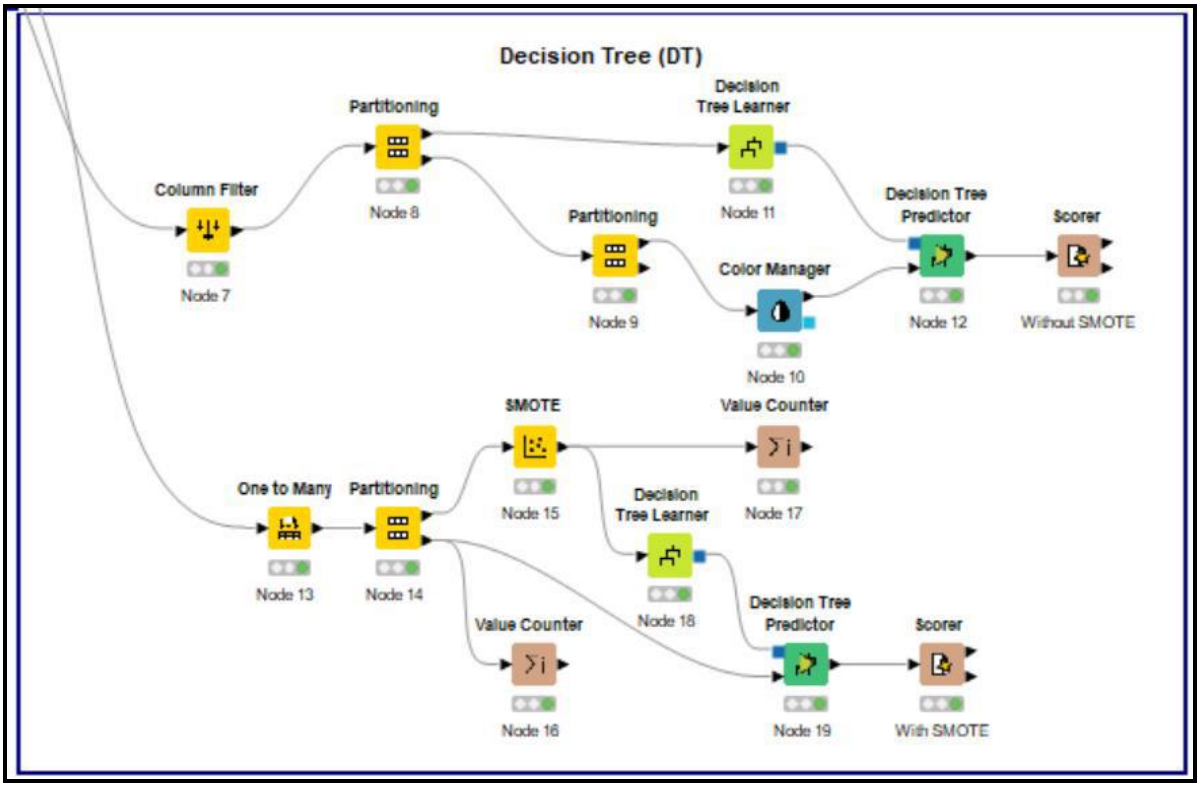
To build a better classifier for predicting the RainTomorrow attribute, I applied both Parameter Optimization, Partitioning, and SMOTE. For Parameter Optimization, I experimented with several techniques, including Brute Force, Hill Climbing, Random Search, and Bayesian Optimization (TPE), and selected the most effective approach for tuning the model's hyperparameters.

Handling unbalanced data was a crucial step, as the RainTomorrow attribute is likely imbalanced (with more "No Rain" than "Rain"). To address this, I applied SMOTE to oversample the minority class (i.e., "Rain") and balance the dataset. This prevents the model from being biased towards the majority class, ensuring better prediction accuracy for both classes.

For Partitioning, I split the dataset into 70% for Training data and 30% for Testing data. I also used validation data to fine-tune the model ensuring that a substantial portion of the data is available for model training. K Fold Cross-validation was also applied to evaluate the model performance across different data subsets, providing a more reliable estimate of its generalization capability. This comprehensive strategy, combined with preprocessing and model tuning, led to a more robust and reliable classifier for predicting RainTomorrow.

B. Classification Techniques

1. Decision Tree Classifier (DT)



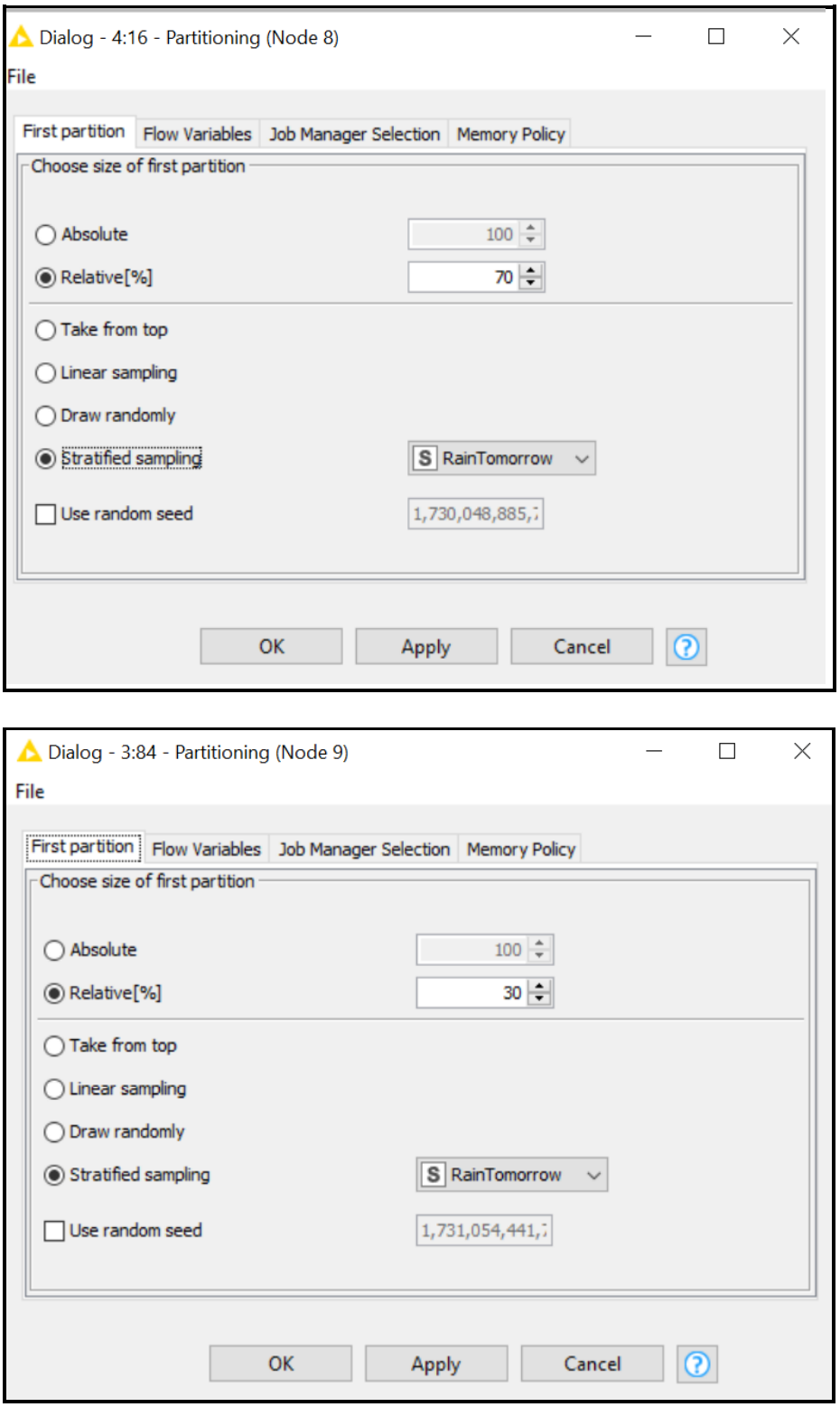


Figure 4 shows the configuration for Partitioning

Figure 4 above shows the configuration for Partitioning Node. In this case, Stratified sampling is chosen because the dataset is imbalanced. In order to ensure that the model is exposed to every class in line with its actual distribution in the dataset, it preserves an equitable representation of each class in both the training and test sets. 70% of the data is used for training and 30% of the data is used for validation. By significantly reducing bias, this method improves the model's ability to generalize new data.

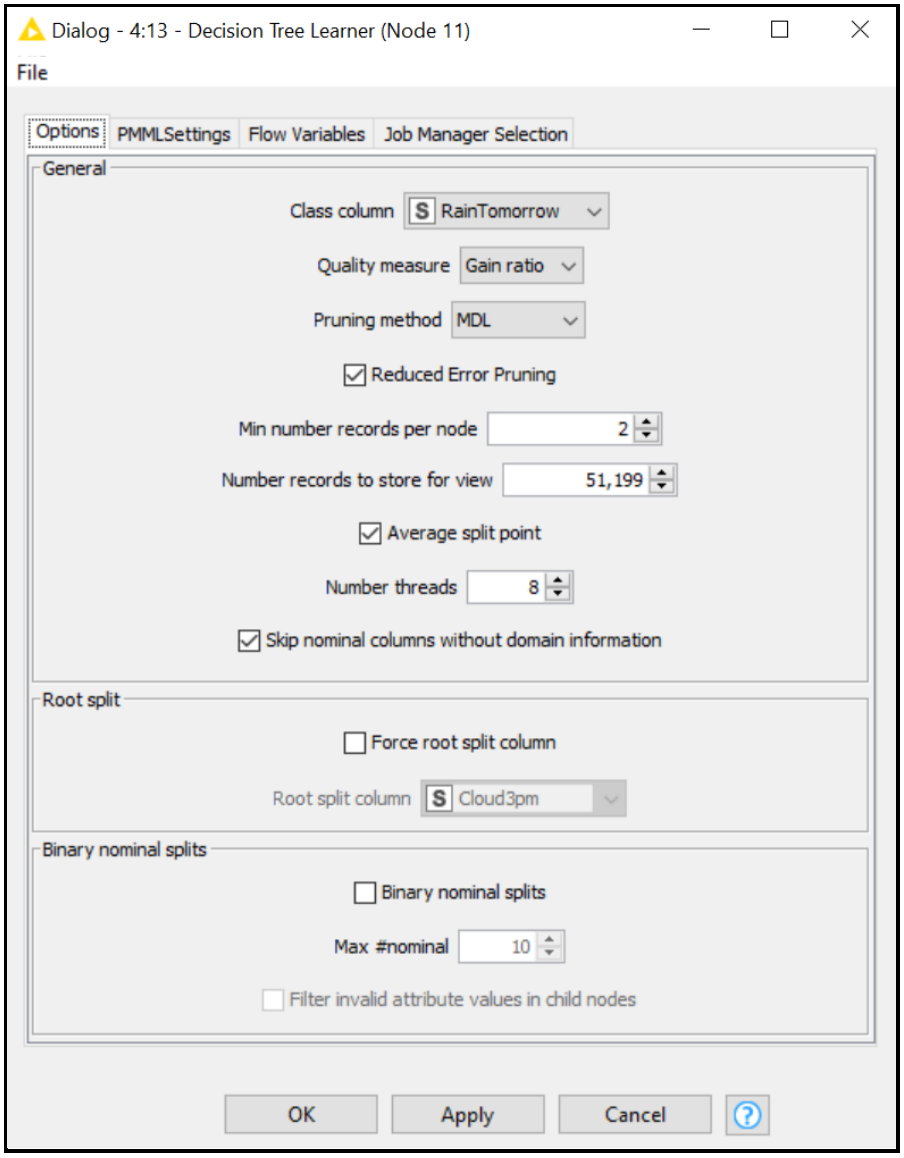


Figure 5 shows the configuration of parameters for Decision Tree Learner node

Based on Figure 5, Pruning is being used and Gain Ratio has been selected as the Quality Measure. Model performance and interpretability can be greatly improved by using the Minimum Description Length (MDL) pruning method and the Quantity Measure. By eliminating branches that add little to the model's prediction ability, the MDL pruning technique reduces overfitting and creates a more straightforward tree structure that more effectively generalizes to unknown data.

Gain Ratio avoids bias towards the data with a large number of classes and offers an equitable evaluation of value importance. In the end, this combination of parameters improves performance metrics like accuracy and F1-score on validation and test datasets by motivating the model to concentrate on variables that are meaningful.

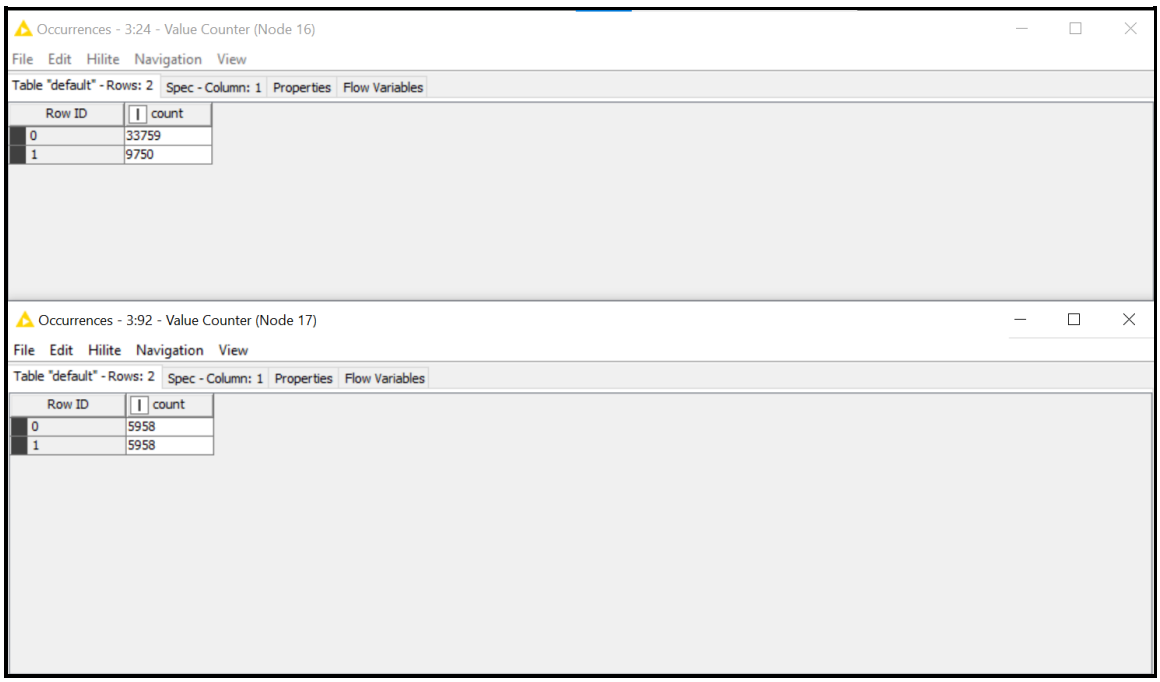


Figure 6 shows the difference in occurrences of 0 and 1 values for RainTomorrow

Figure 6 illustrates the difference in the distribution of values before and after applying SMOTE, respectively. This helps to address class imbalance, leading to a better-performing classifier.

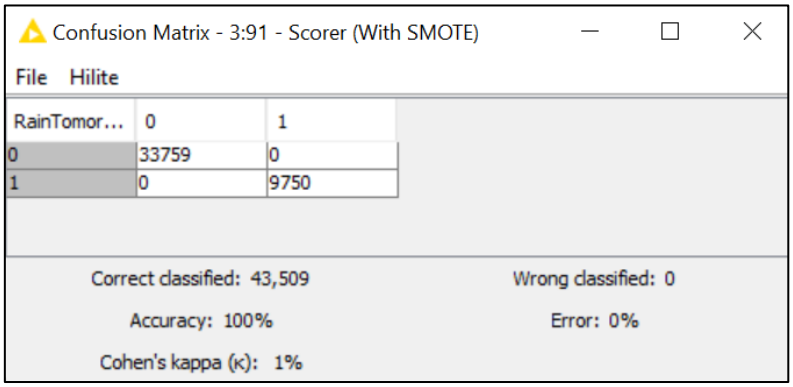


Figure 7 shows the Confusion Matrix for Decision Tree Classifier with SMOTE

As shown in Figure 7, the confusion matrix reports an accuracy of 100% for the Decision Tree with SMOTE applied, with 0% error. This result is not a genuine measure of performance. Reviewing the workflow afterwards, I found that the One-to-Many node was applied to RainTomorrow with the “remove source columns” option switched off, so exact 0/1 copies of the target were passed to the model as input columns (target leakage). The tree could therefore read the answer directly from its inputs. The figure should be treated as an error in my workflow and not as evidence of a good model.

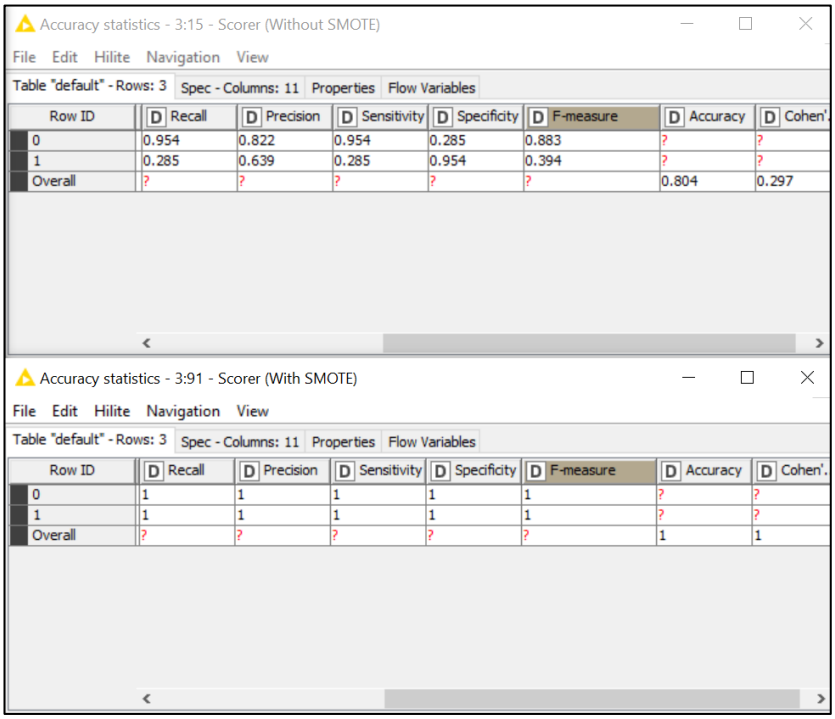
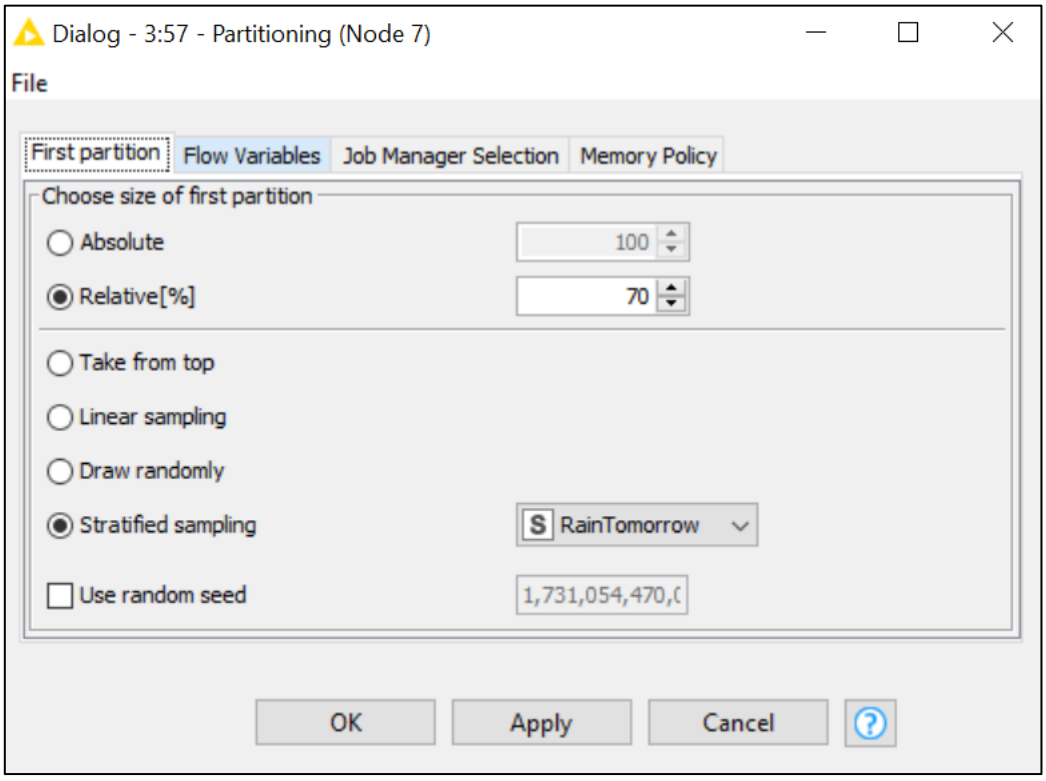


Figure 8 shows the Accuracy Statistics for Decision Tree classifier, with and without SMOTE

2. K-Nearest Neighbor Classifier (KNN)



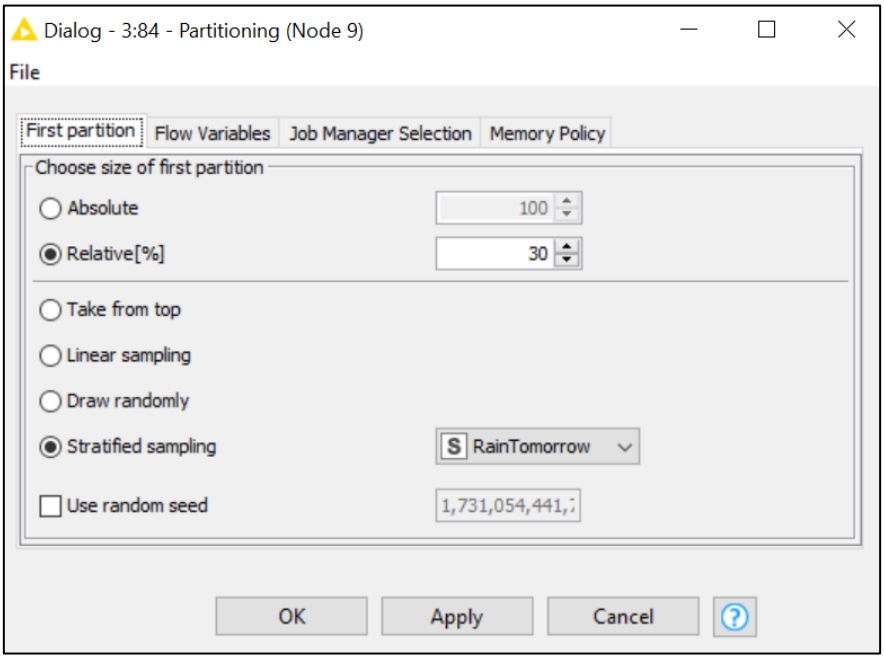
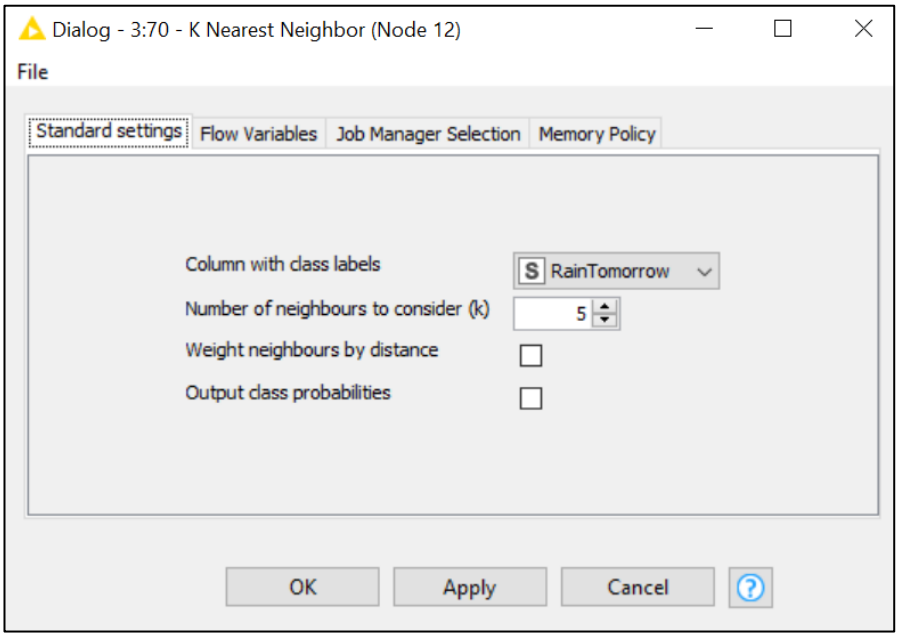


Figure 9 shows the configuration for Partitioning

Figure 9 shows the configuration for Partitioning Node. In this case, Stratified sampling is chosen because the dataset is imbalanced. 70% of the data is used for training and 30% of the data is used for validation.



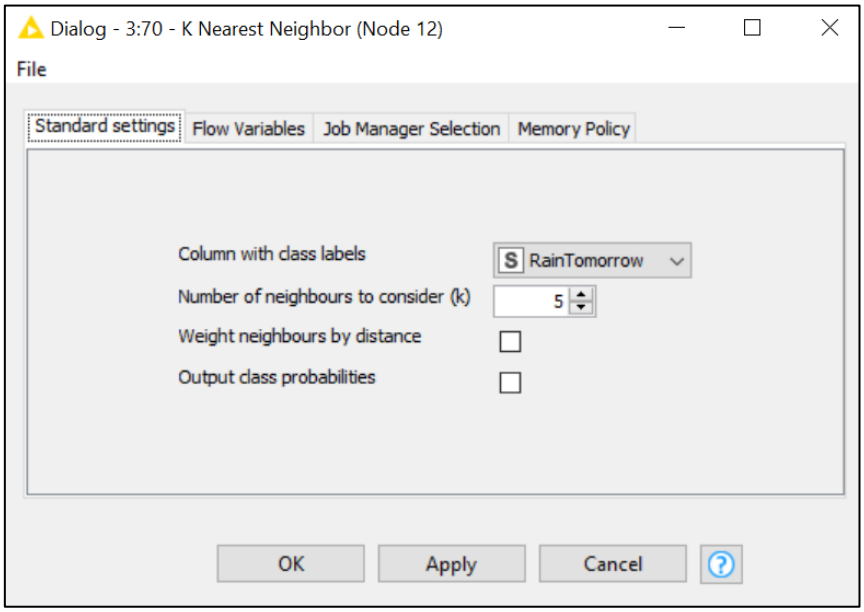


Figure 10 shows the configuration of parameters for KNN

As shown in Figure 10, 5 is the chosen value for k for the KNN configuration parameters because it strikes a good balance between bias and variance. A smaller k could lead to overfitting by making the model too sensitive to noise, while a larger k might oversmooth the data and miss important patterns. k = 5 is commonly used as it tends to provide stable performance, generalizing well across a variety of datasets without being too sensitive or overly simplistic.

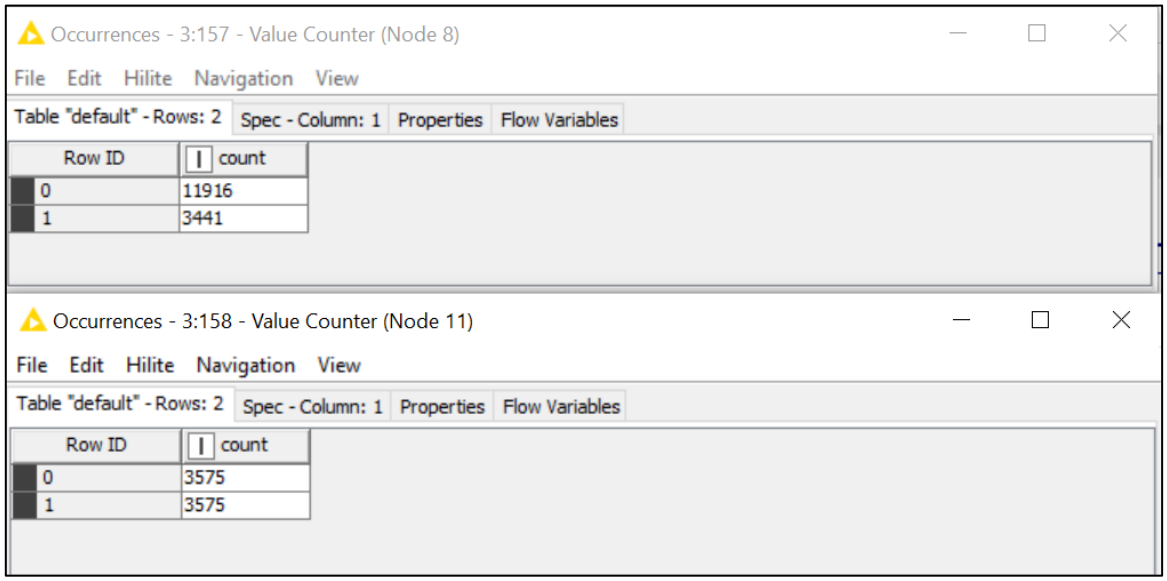


Figure 11 shows the difference in occurrences of 0 and 1 values for RainTomorrow

Figure 11 illustrates the difference in the distribution of values before and after applying SMOTE, respectively.

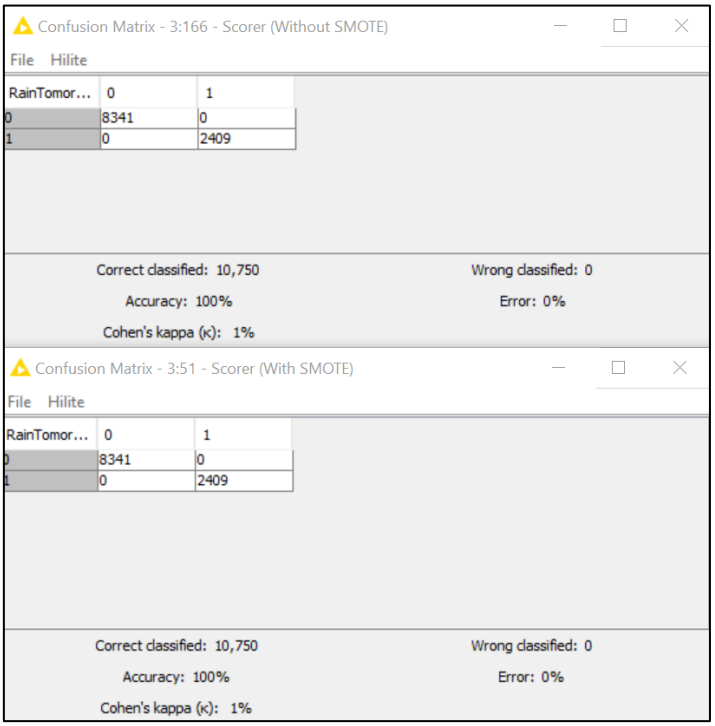


Figure 12 shows the Confusion Matrix for KNN with and without SMOTE

As shown in Figure 12, the confusion matrix reports 100% accuracy for KNN both with and without SMOTE. The identical result in both cases is itself a warning sign, because SMOTE would normally change the outcome. The cause is the same target leakage described for the Decision Tree: the One to Many node left copies of RainTomorrow among the input columns, so the model matched each row to its own answer. The 100% figure therefore reflects the leak and not the model's ability to predict rain.

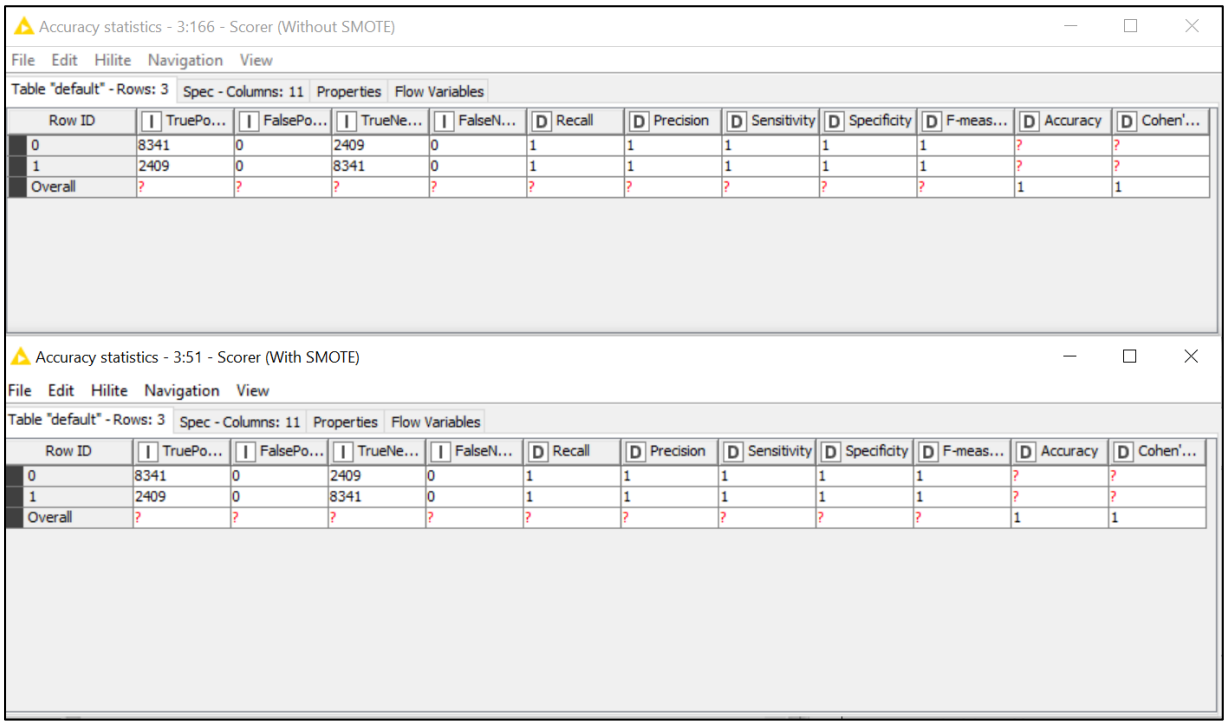
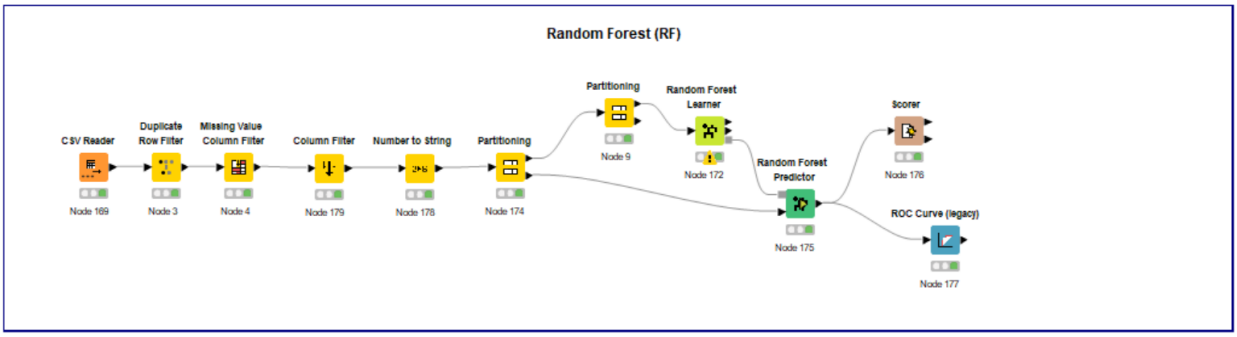
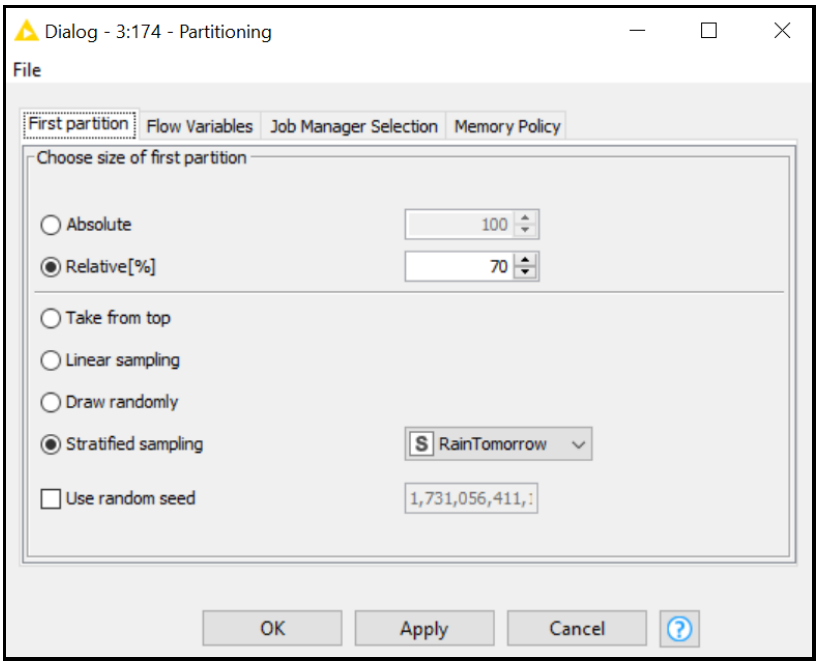


Figure 13 shows the Accuracy Statistics for KNN classifier, with and without SMOTE

3. Random Forest (RF)





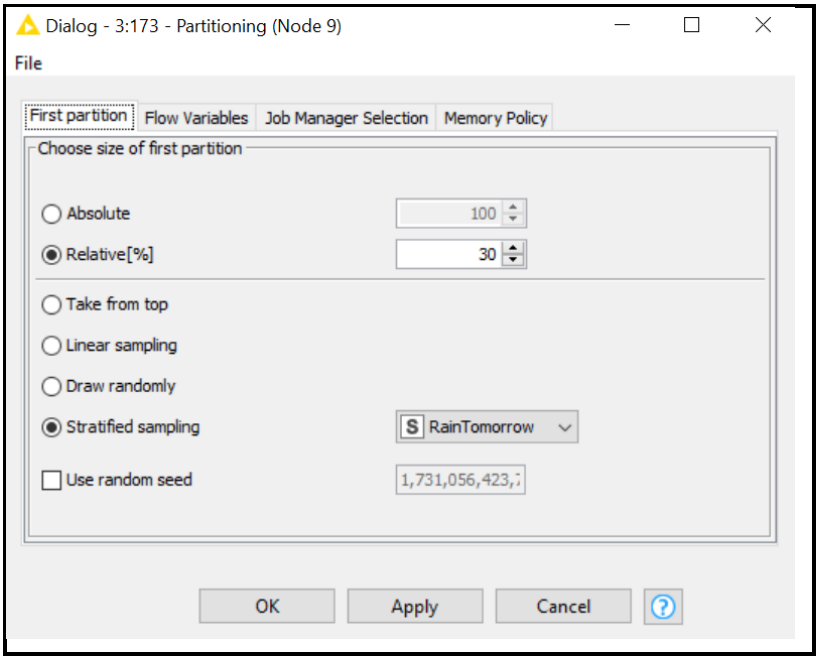


Figure 14 shows the configuration for Partitioning

Figure 14 shows the configuration for Partitioning Node. In this case, Stratified sampling is chosen because the dataset is imbalanced. 70% of the data is used for training and 30% of the data is used for validation.

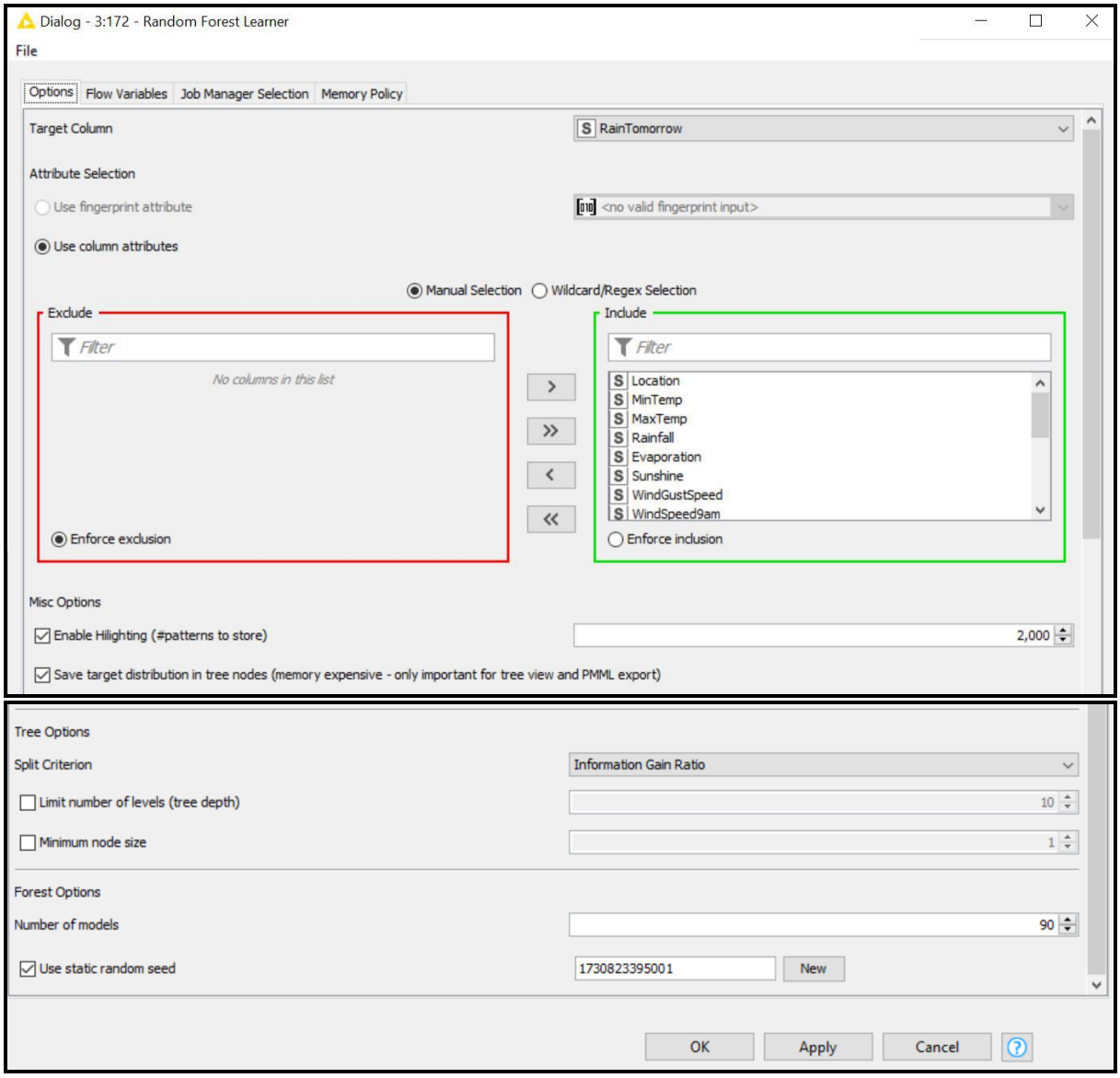


Figure 15 shows the configuration of parameters for RF

As shown in Figure 15, I decided to use Information Gain Ratio instead of Information Gain or the Gini Index because it gave better results. Information Gain Ratio helps avoid the problem of favoring features with too many categories, which can happen with Information Gain. The model's accuracy is higher when using Information Gain Ratio.

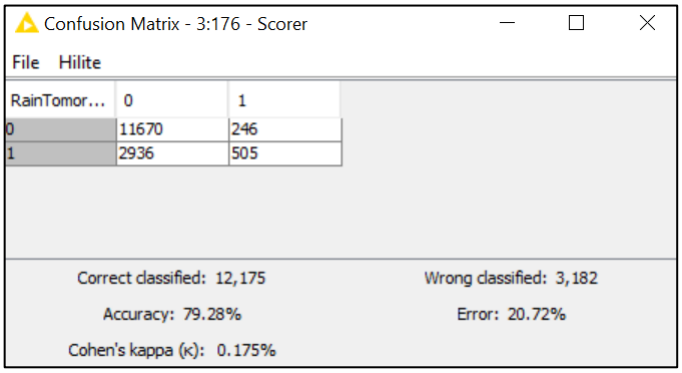


Figure 16 shows the Confusion Matrix for RF

As shown in Figure 16, the Confusion Matrix indicates that this model's accuracy is 79.28%. This means that the model correctly predicted the target variable in 79.28% of the cases, while it made incorrect predictions 20.72% of the time. This suggests that the model performs reasonably well, correctly classifying most instances. However, the error rate indicates that there is still some room for improvement, as the model occasionally misclassifies the data.

For context, always predicting “No Rain” would be correct for about 78% of the validation rows (11,916 of 15,357), so an accuracy of 79.28% is only slightly better than a baseline that ignores the weather entirely.

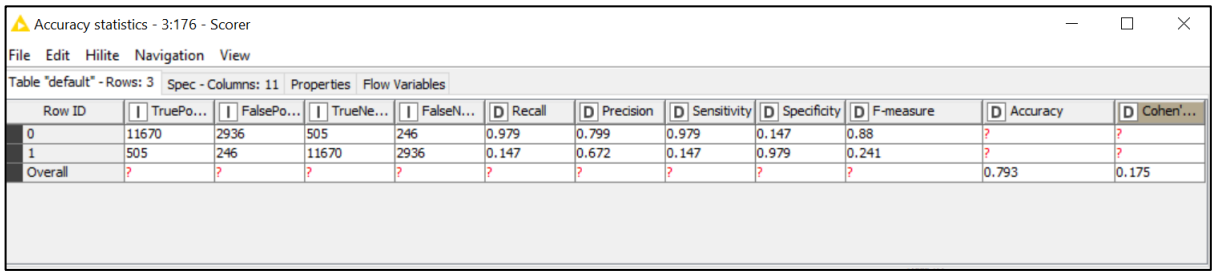


Figure 17 shows the Accuracy Statistics for RF

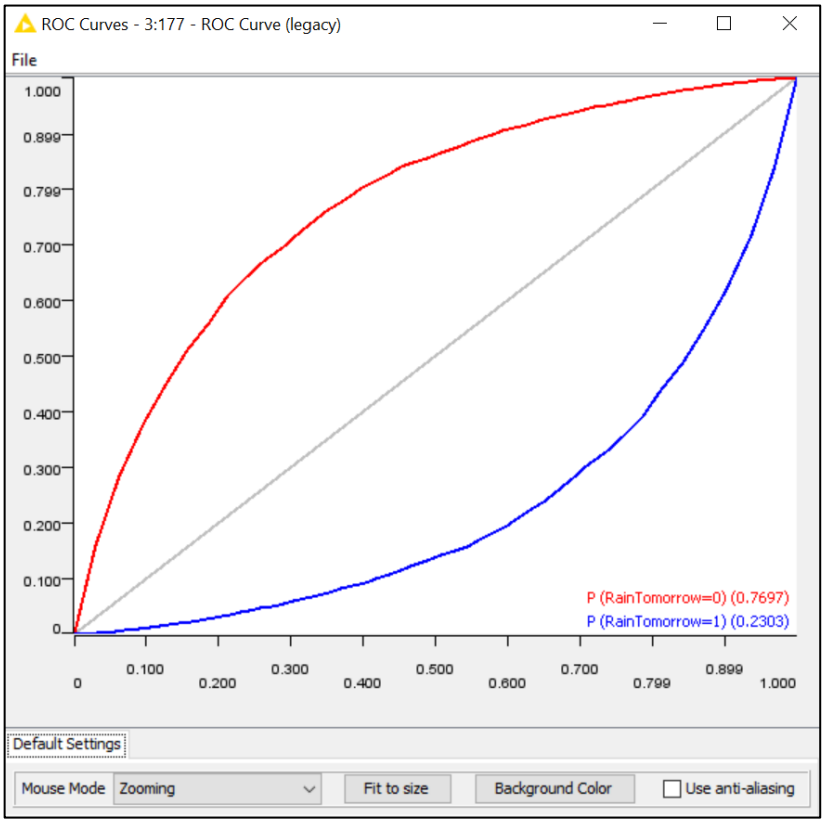
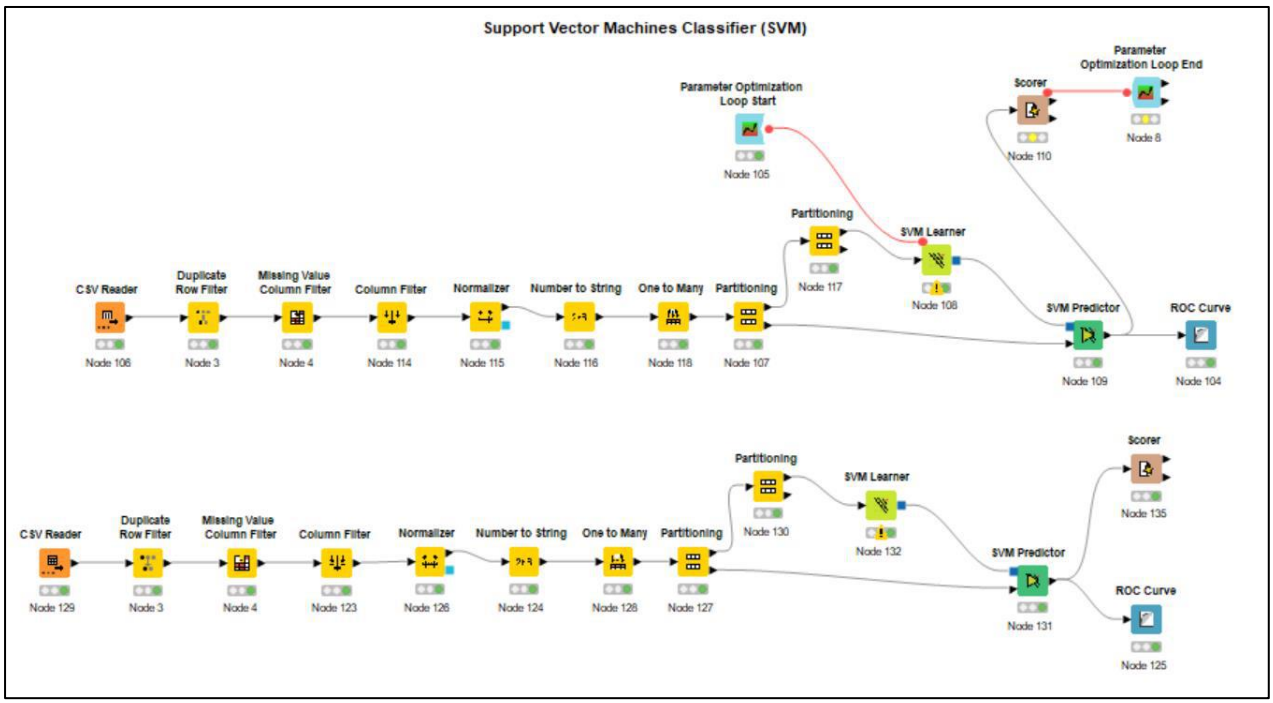


Figure 18 shows the ROC curve for RF

As shown in Figure 18, the prediction probabilities for RainTomorrow indicate that the model is more confident about predicting No Rain (RainTomorrow = 0) with a probability of 0.7697, while it assigns a lower probability of 0.2303 to Rain (RainTomorrow = 1). This suggests that the model tends to predict "No Rain" more often.

4. Support Vector Machines Classifier (SVM)



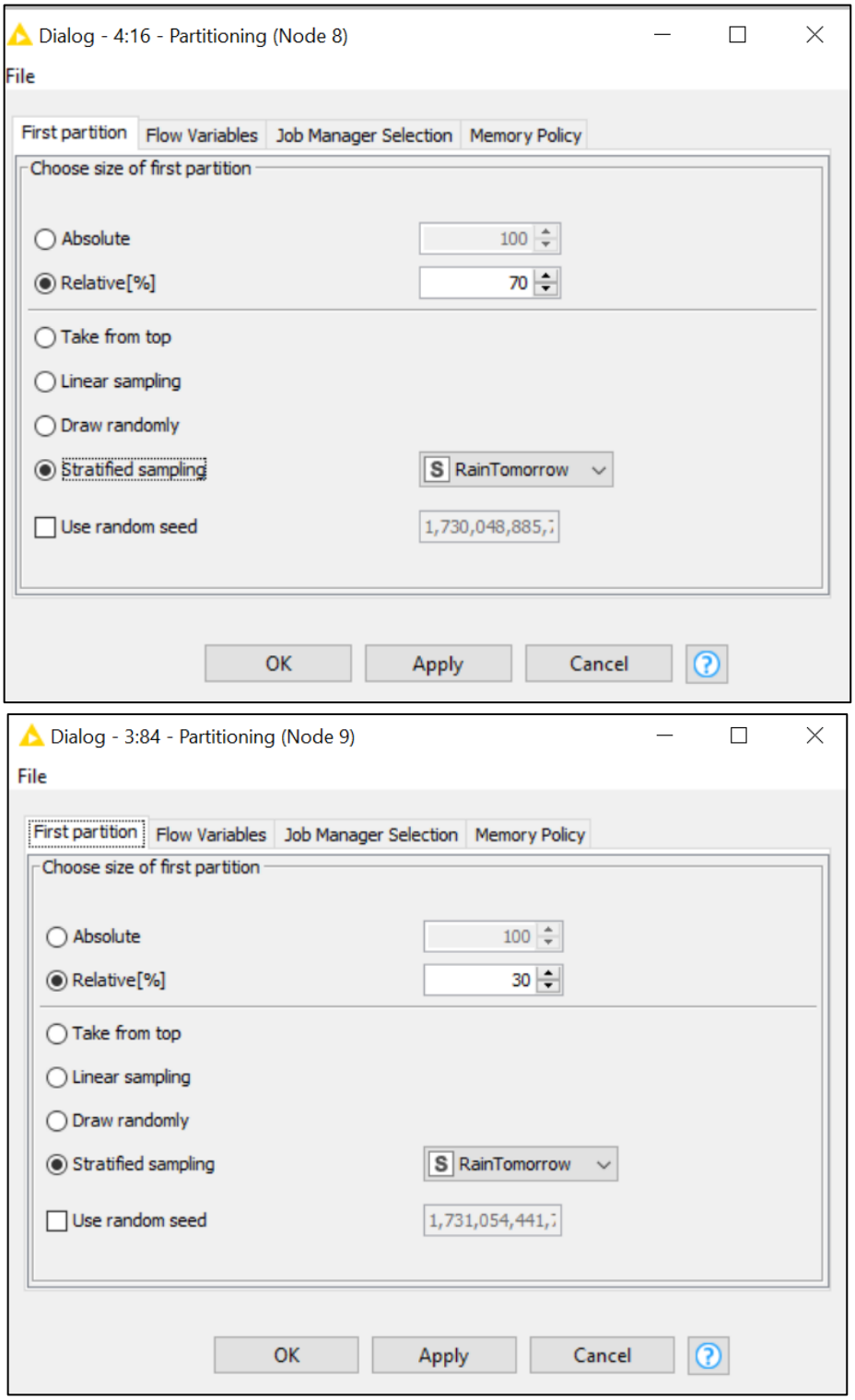


Figure 19 shows the configuration for Partitioning

Figure 19 shows the configuration for Partitioning Node. In this case, Stratified sampling is chosen because the dataset is imbalanced. 70% of the data is used for training and 30% of the data is used for validation.

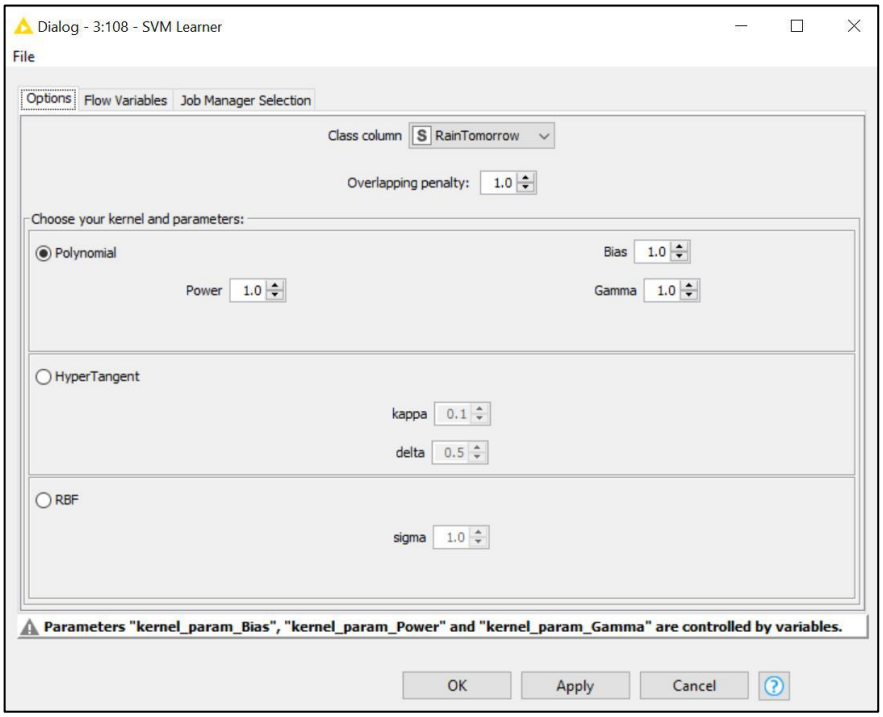
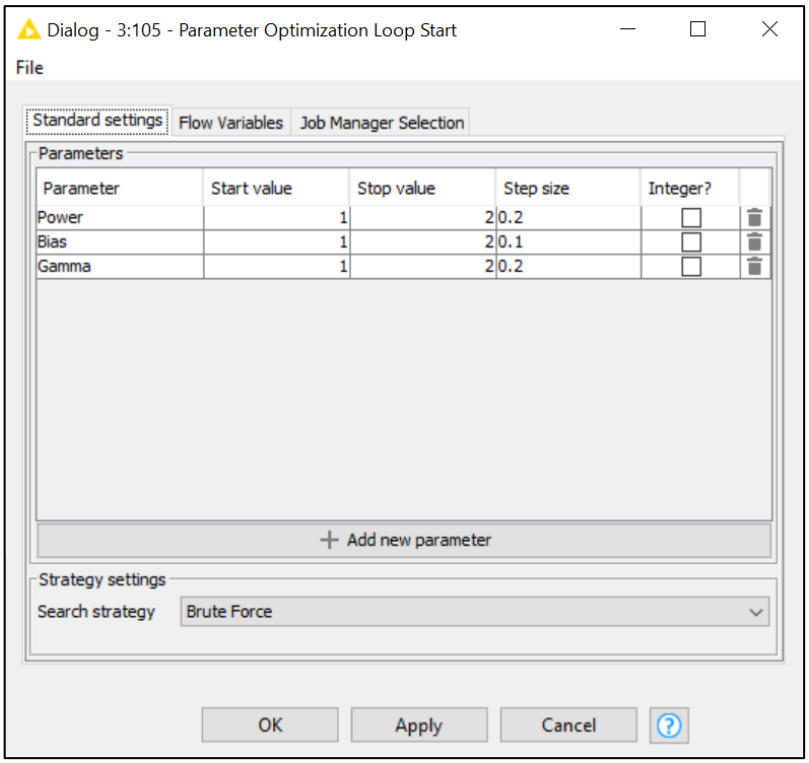


Figure 20 shows the parameters for SVM

As shown in Figure 20, Polynomial kernel is a good choice for SVM because it performs better than the HyperTangent and Radial and RBF kernels on this dataset. The Polynomial kernel is well-suited for capturing more complex relationships between features, and in this case, it provided better classification results.



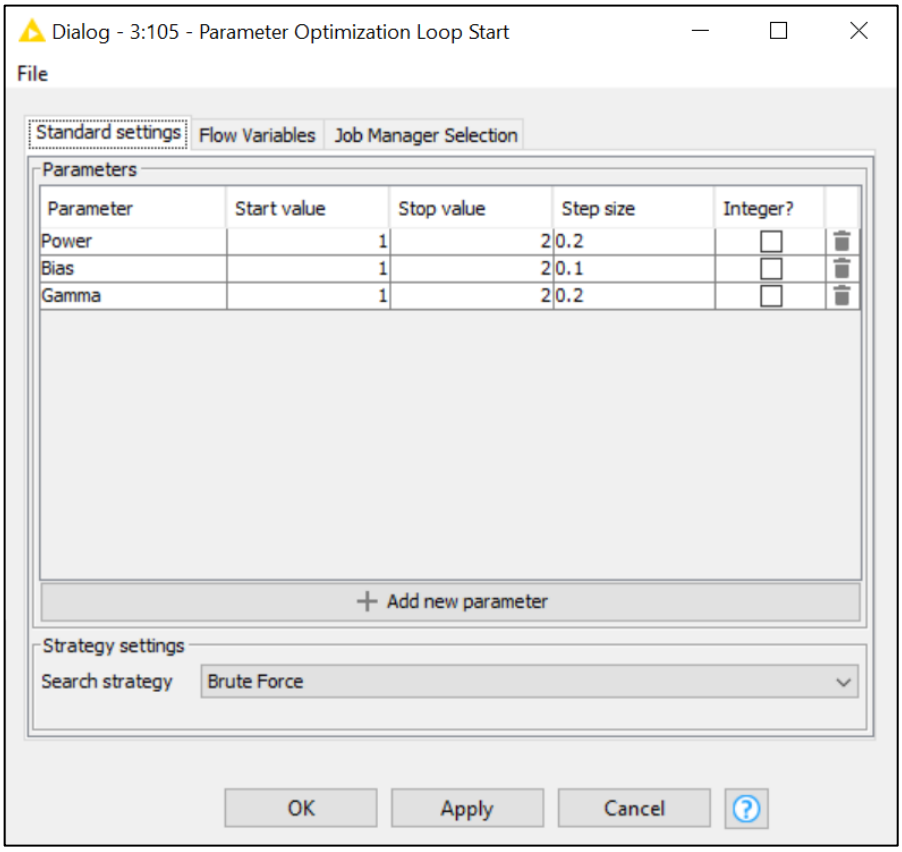


Figure 21 shows the Parameter Optimization for SVM

For parameter optimization, Brute Force search strategy was chosen because it systematically tests all possible combinations to ensure that the best set of values is found. While this approach can be computationally intensive, it is effective for thoroughly exploring the hyperparameter space, especially when the range of parameters is manageable.

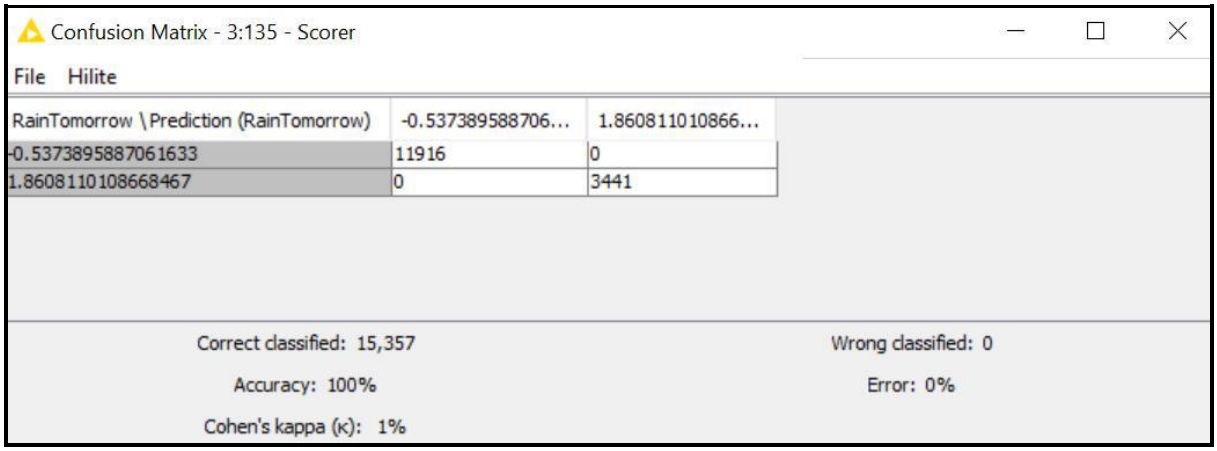


Figure 22 shows the Confusion Matrix for SVM

The SVM with a polynomial kernel also reports 100% accuracy (15,357 validation rows classified correctly, Cohen's kappa of 1), but the ROC curve lies exactly on the diagonal of a random classifier, which is impossible for a genuinely perfect model. Together with the standardized class labels (-0.537 and 1.860) in the confusion matrix, this shows that a numeric copy of RainTomorrow was present among the model inputs. This is the same target leakage

caused by the One-to-Many node described for the Decision Tree and KNN, so the SVM result is not a reliable measure of performance.

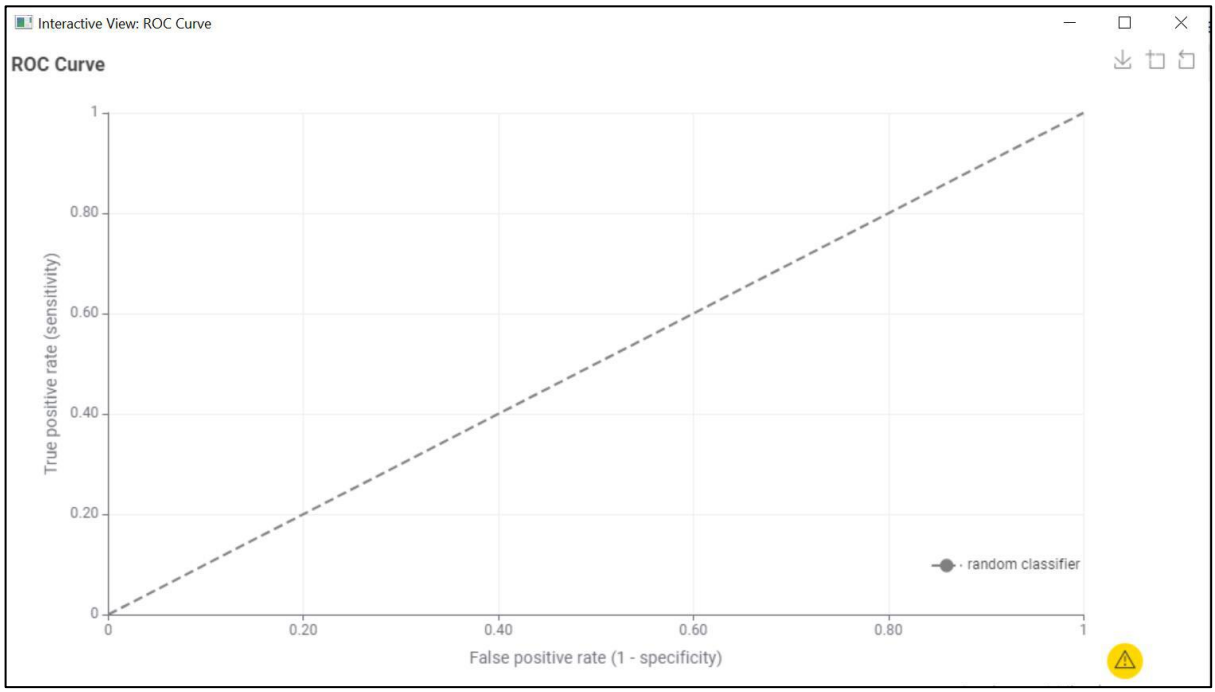
Accuracy statistics - 3:135 - Scorer

File Edit Hilite Navigation View

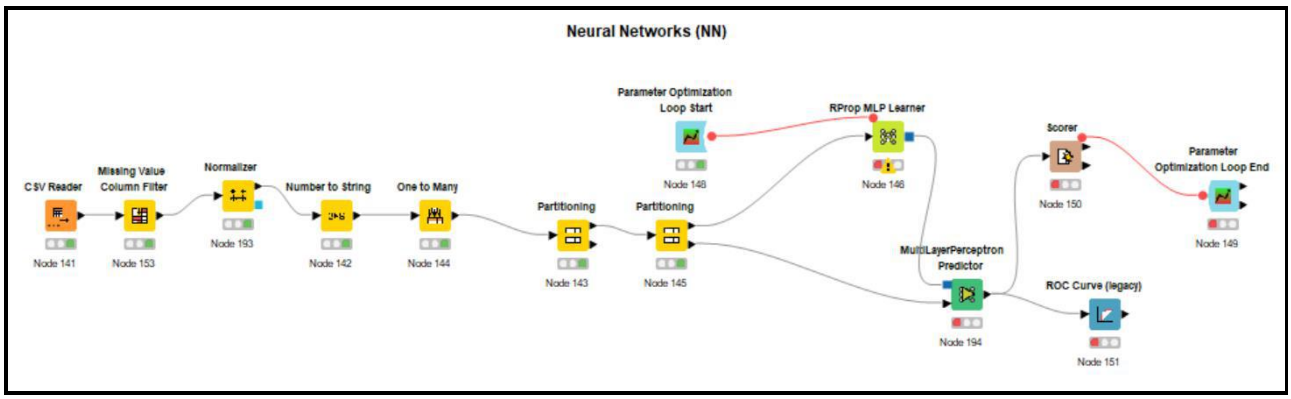
Table "default" - Rows: 3 | Spec - Columns: 11 | Properties | Flow Variables

Row ID	TruePo...	FalsePo...	TrueNe...	FalseN...	Recall	Precision	Sensitivity	Specificity	F-meas...	Accuracy	Cohen'...
-0.53738958...	11916	0	3441	0	1	1	1	1	1	?	?
1.860811010...	3441	0	11916	0	1	1	1	1	1	?	?
Overall	?	?	?	?	?	?	?	?	?	1	1

Figure 23 shows the Accuracy Statistics for SVM



5. Neural Network (NN)



Other Classifiers

1. K-Fold Cross Validation (KFCV)

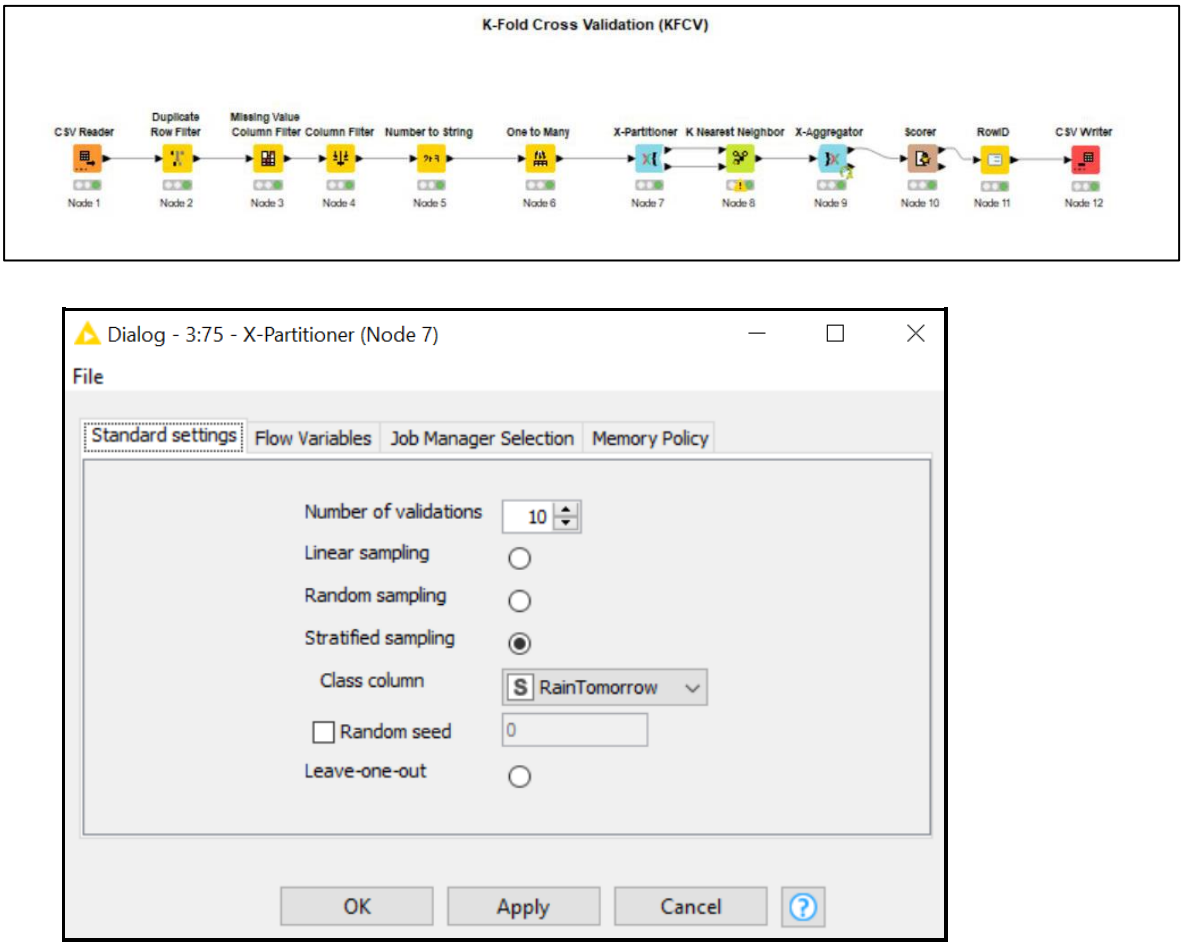


Figure 25 shows the parameters for KFCV

10-fold cross-validation was chosen because it provides a more reliable estimate of the model's performance by training and testing the model on different subsets of the data, reducing bias and overfitting. This method uses the entire dataset for both training and testing, ensuring better generalization to unseen data. Data partitioning, on the other hand, can be less reliable because the performance depends on a single, potentially unrepresentative split of the data, which might lead to biased results.

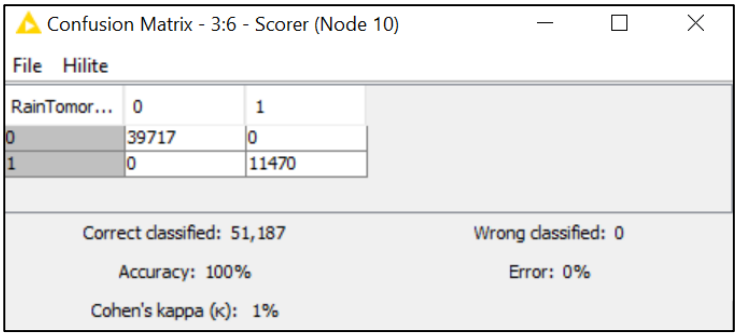


Figure 26 shows the Confusion Matrix for KFCV

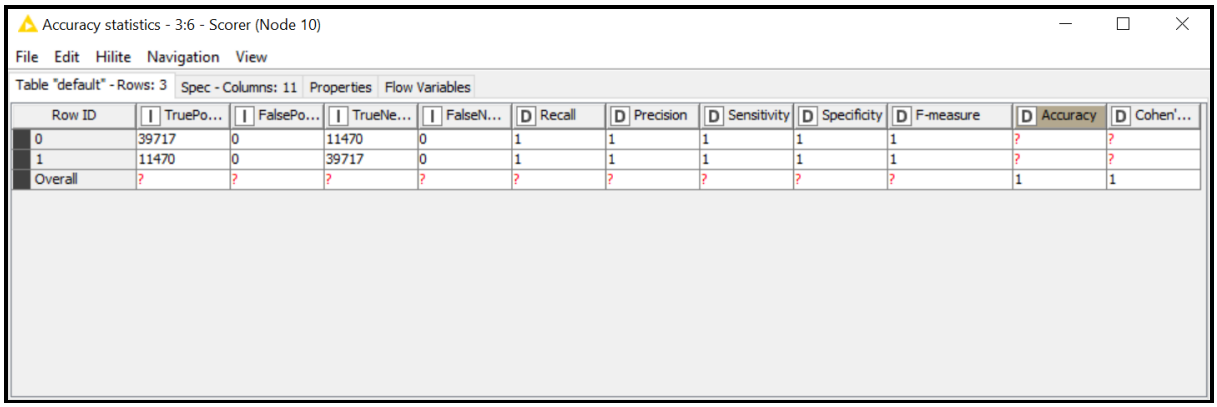


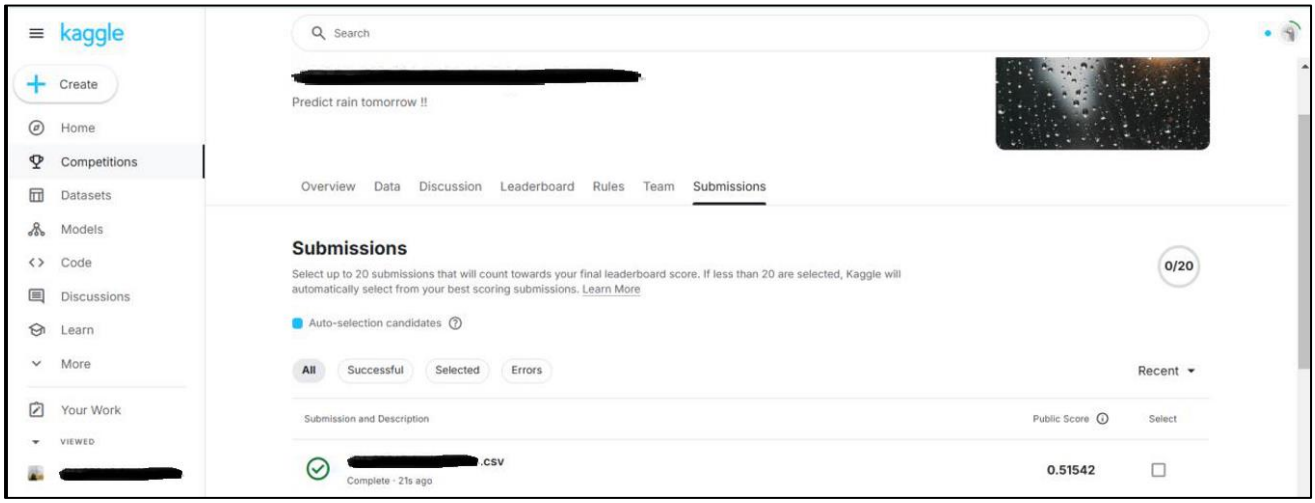
Figure 27 shows the Accuracy Statistics for KFCV

Conclusion

I selected K-Nearest Neighbors (KNN) with k = 5 and SMOTE because it reported the highest validation accuracy (100%), and because it is simple and makes no strong assumptions about the data. Stratified sampling was used during partitioning to keep the Rain and No Rain proportions the same in both partitions. However, this selection is not supported by the evidence. The 100% accuracy came from target leakage caused by the One-to-Many node, which left copies of RainTomorrow among the model inputs.

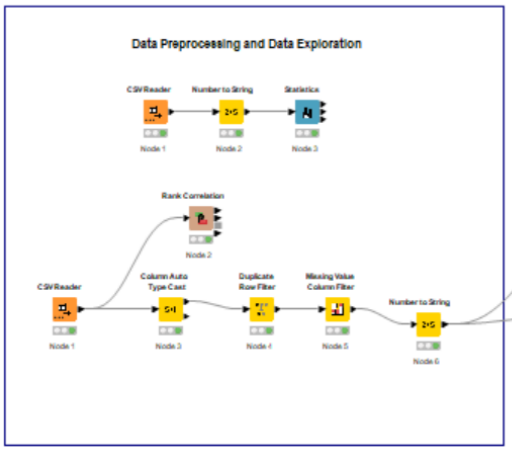
The Kaggle score of 0.51542 confirms this, since it is close to the 0.5 expected from random guessing. In a corrected workflow, the One-to-Many node would exclude RainTomorrow, normalization and encoding would be fitted on the training partition only, and SMOTE would be applied to the training partition only. The classifiers would then be compared again before choosing the best one.

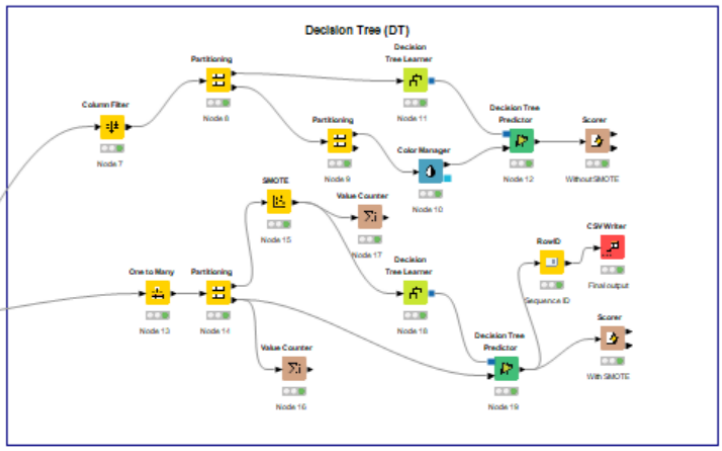
Kaggle Submission Score

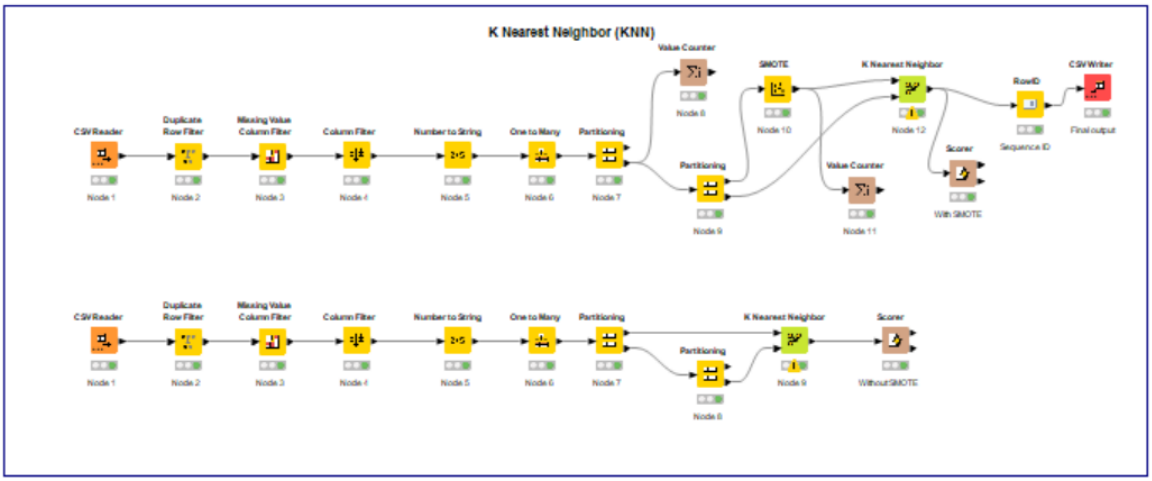


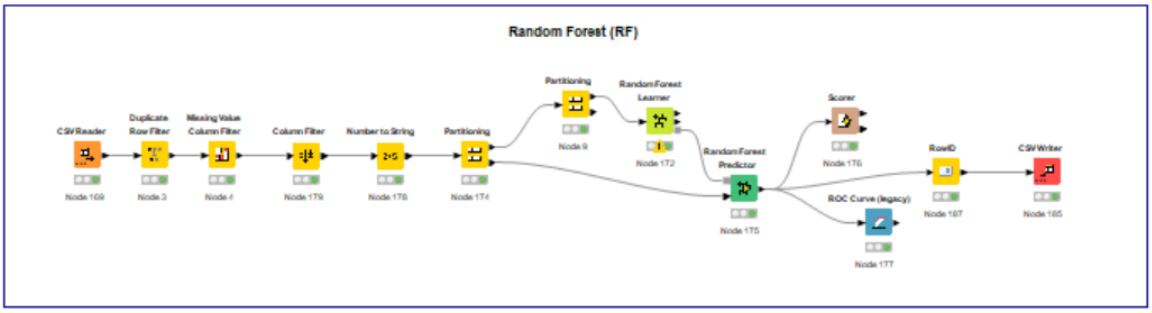
Appendix

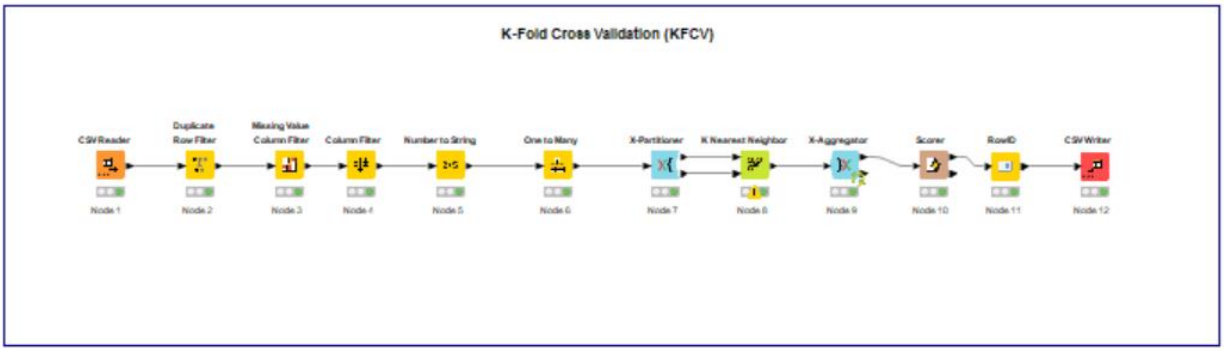
Student Name: Brunda Vanshini Katapalli

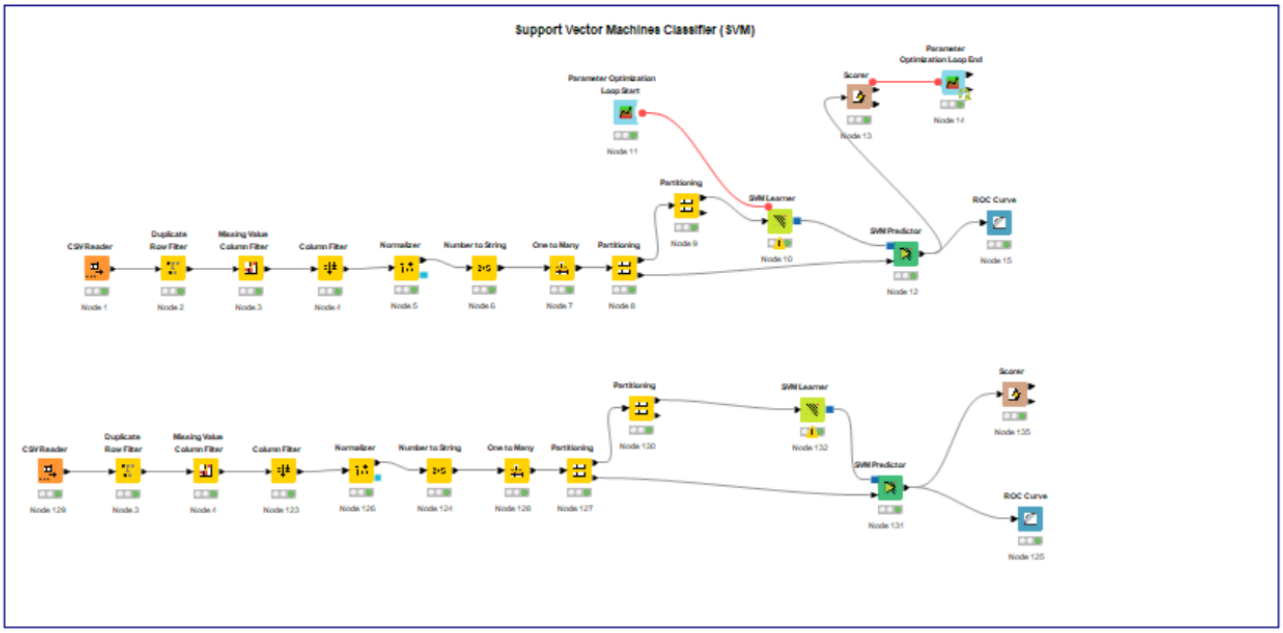


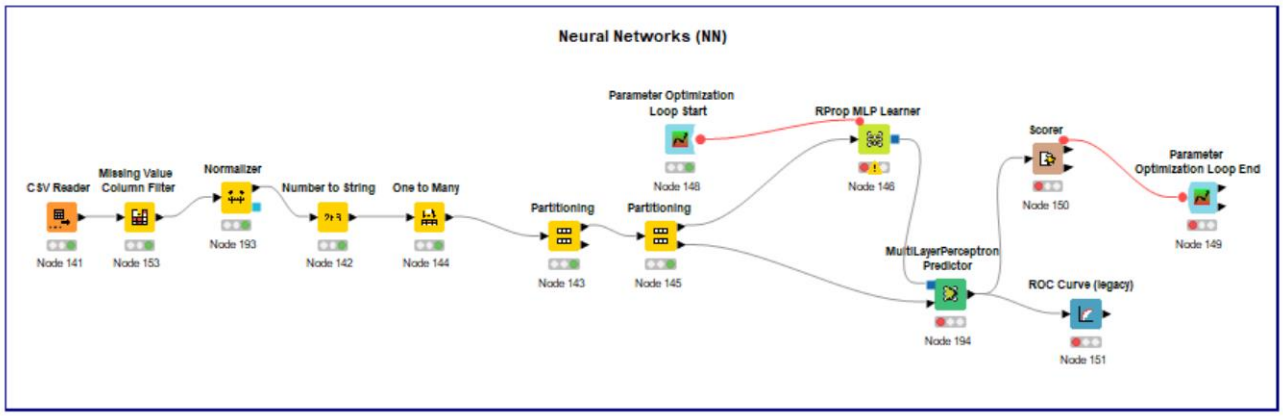









Support Vector Machines Classifier (SVM)
Parameter Optimization Loop Start
Node 11
Scorer
Node 13
Parameter Optimization Loop End
Node 14
Partitioning
Node 9
SVM Learner
Node 10
ROC Curve
Node 15
SVM Predictor
Node 12
CSV Reader
Node 1
Duplicate Row Filter
Node 2
Missing Value Column Filter
Node 3
Column Filter
Node 4
Normalizer
Node 5
Number to String
Node 6
One to Many
Node 7
Partitioning
Node 8
Partitioning
Node 130
SVM Learner
Node 132
Scorer
Node 135
CSV Reader
Node 129
Duplicate Row Filter
Node 3
Missing Value Column Filter
Node 4
Column Filter
Node 123
Normalizer
Node 126
Number to String
Node 124
One to Many
Node 128
Partitioning
Node 127
SVM Predictor
Node 131
ROC Curve
Node 125


Neural Networks (NN)
Parameter Optimization Loop Start
Node 148
RProp MLP Learner
Node 146
Scorer
Node 150
Parameter Optimization Loop End
Node 149
CSV Reader
Node 141
Missing Value Column Filter
Node 153
Normalizer
Node 193
Number to String
Node 142
One to Many
Node 144
Partitioning
Node 143
Partitioning
Node 145
MultiLayerPerceptron Predictor
Node 194
ROC Curve (legacy)
Node 151